El Gran Torneo Pokémon

Guía de lectura, arquitectura, estructuras de datos,
diagramas y verificación de requisitos para la defensa oral

Programación I / Algoritmos I — Universidad de Carabobo
C (ISO C99), gcc -std=c99 -Wall -Wextra

26 de octubre de 2026

Índice

1. Cómo leer el proyecto: guía ordenada **2**

2. Arquitectura: los módulos **2**

3. Estructuras de datos utilizadas **4**

3.1. Respuesta directa: ¿pilas? ¿listas? ¿colas? 4

3.2. Inventario y justificación 4

4. Diagramas de flujo **4**

4.1. Carga de la Pokédex (`pokedex_cargar`) 5

4.2. Combate (`combate_ejecutar`) 6

4.3. Backtracking de formación de equipos (`bt_rec`) 6

4.4. Máquina de estados del torneo 7

4.5. Flujo resumido de cada módulo 7

5. Modelo entidad-relación **8**

6. Cumplimiento del documento de requerimientos **9**

7. Guion de defensa: preguntas probables **10**

8. Comandos útiles durante la defensa **11**

1. Cómo leer el proyecto: guía ordenada

El proyecto son 10 módulos (par `.c`/`.h`), una cabecera de constantes y la batería de pruebas (5 011 líneas en `src/`). El orden recomendado de lectura va de los datos a la lógica, y de lo simple a lo compuesto; cada paso indica qué buscar.

1. `src/constantes.h` (51 líneas) — el "vocabulario" del dominio: 150 especies, 32 entrenadores, 18 tipos, equipo de 6, niveles 1–100, 64 combates, buffers y rutas. Sin código: solo decisiones en un lugar.
2. `src/pokedex.h/.c` — la fuente de datos. El struct `Especie`, la carga estricta desde archivo (`pokedex_cargar`) y la inmutabilidad (todo es `const`).
3. `src/tipos.h/.c` — el catálogo de 18 tipos y la matriz de efectividad 18×18 (`static` en el `.c`).
4. `src/entrenador.h/.c` — el struct `Entrenador`, el registro de 32 y el id único.
5. `src/equipo.h/.c` — el módulo más denso: el `Ejemplar` como nodo de lista enlazada, la fórmula de estadísticas (D2), la separación especie/ejemplar y el backtracking (`bt_rec`).
6. `src/combate.h/.c` — el motor: fórmula de daño (D1), orden por velocidad, KO, empates y anti-empate; el patrón de callback (`CombateSeleccionar`).
7. `src/torneo.h/.c` — la máquina de estados: grupos A–H, calendario 1–48, clasificación con desempates y bracket 49–64.
8. `src/resultados.h/.c` — carga por teclado/archivo y el catálogo completo de validación de resultados.
9. `src/archivos.h/.c` — persistencia: guardar/cargar entrenadores, resultados y clasificación.
10. `src/validacion.h/.c` — lectura segura con reintentos y manejo de EOF (ninguna función llama `exit`).
11. `src/main.c` (794 líneas) — solo flujo y menú: 17 funciones `static` privadas que orquestan los módulos. Sin lógica de negocio.
12. `tests/` — la evidencia: `run_tests.sh`, 20 casos de menú y 2 probes C (efectividad e integración completa del torneo).

Regla de oro del proyecto: la Pokédex jamás se escribe; los datos de las especies son inmutables y los ejemplares copian lo que necesitan.

2. Arquitectura: los módulos

Cada módulo es un TDA: un tipo de dato + sus operaciones, con la cabecera como contrato público y la implementación encapsulada en el `.c`. Los identificadores siguen la convención *modulo_ - accion*.

Módulo (.c/.h)	**Líneas**	**Responsabilidad y funciones públicas clave**
`constantes.h`	51	Dominio en un lugar: `POKEDEX_MAX`, `MAX_ENTRENADORES`, `CANT_TIPOS`, `MAX_EQUIPO`, `NIVEL_MIN/MAX`, `TOTAL_COMBATES`, rutas.

`pokedex`	308+91	`pokedex_cargar` (void, +ok), `pokedex_buscar_numero/nombre` (puras), `pokedex_mostrar_todas/especie`. Posee `Especie` y `Pokedex[150]`.
`tipos`	285+71	`tipos_inicializar`, `tipos_es_valido`, `tipos_a_texto`, `tipos_multiplicador` (pura). Posee el enum `Tipo` y la matriz 18×18.
`entrenador`	80+76	`entrenador_registrar` (void, +ok), `entrenador_buscar`, `entrenador_mostrar_todos`. Posee `Entrenador` y `RegistroEntrenadores[32]`.
`equipo`	467+195	`equipo_crear_ejemplar`, `equipo_agregar_ejemplar`, `equipo_validar`, `equipo_liberar`, `equipo_asignar`, `equipo_mostrar`, `equipo_siguiente_id`, `equipo_formar_backtracking` (void, +ok). Posee `Ejemplar` (lista enlazada), `RestriccionesEquipo` y el backtracking.
`combate`	337+96	`combate_calcular_danio` (pura), `combate_ataca_primero` (pura), `combate_ejecutar` (void, +ok). Posee `ResultadoCombate` y el callback `CombateSeleccionar`.
`torneo`	818+219	`torneo_armar_grupos`, `torneo_participantes`, `torneo_aplicar_resultado`, `torneo_ordenar_grupos`, `torneo_clasificados`, `torneo_clasificacion_grupo` y 5 `torneo_mostrar_*`. Posee `Combate[64]` y `Torneo`.
`resultados`	398+77	`resultados_cargar_teclado/archivo` (void, +ok), `resultados_validar` (void, +ok), `resultados_mostrar_pendientes`.
`archivos`	340+92	Persistencia: 5 funciones `archivos_cargar/guardar_*` (void, +ok). Formatos D8.
`validacion`	134+82	`validar_leer_entero/_msg`, `validar_leer_cadena` (void, +ok), `validar_separar_campos` (void, +n). Ninguna llama `exit`.
`main`	794	Menú de 12 opciones y despacho; 17 ayudantes `static`; `main` solo orquesta y libera memoria.

Convención de funciones (especificada en `openspec/specs/convencion-funciones/spec.md`, RF-CNV-01..03): toda función no-`void` devuelve *un solo* resultado y no modifica parámetros; las operaciones con múltiples salidas son procedimientos `void` con parámetros por referencia (patrón `void f(..., bool *exito)`).

3. Estructuras de datos utilizadas

3.1. Respuesta directa: ¿pilas? ¿listas? ¿colas?

- **Listas: SÍ** — lista enlazada simple de `Ejemplar` (campo `struct Ejemplar *siguiente`), con inserción al frente y `malloc`/`free` gestionados por el módulo `equipo`. Es la estructura dinámica que exige el enunciado.
- **Pilas: NO explícitamente** — se usa *la pila del sistema* (stack de llamadas) como pila implícita de la recursión del backtracking: cada llamada recursiva guarda su estado y el retroceso (backtracking) "desapila" al retornar.
- **Colas: NO** — ninguna operación del torneo requiere orden FIFO.

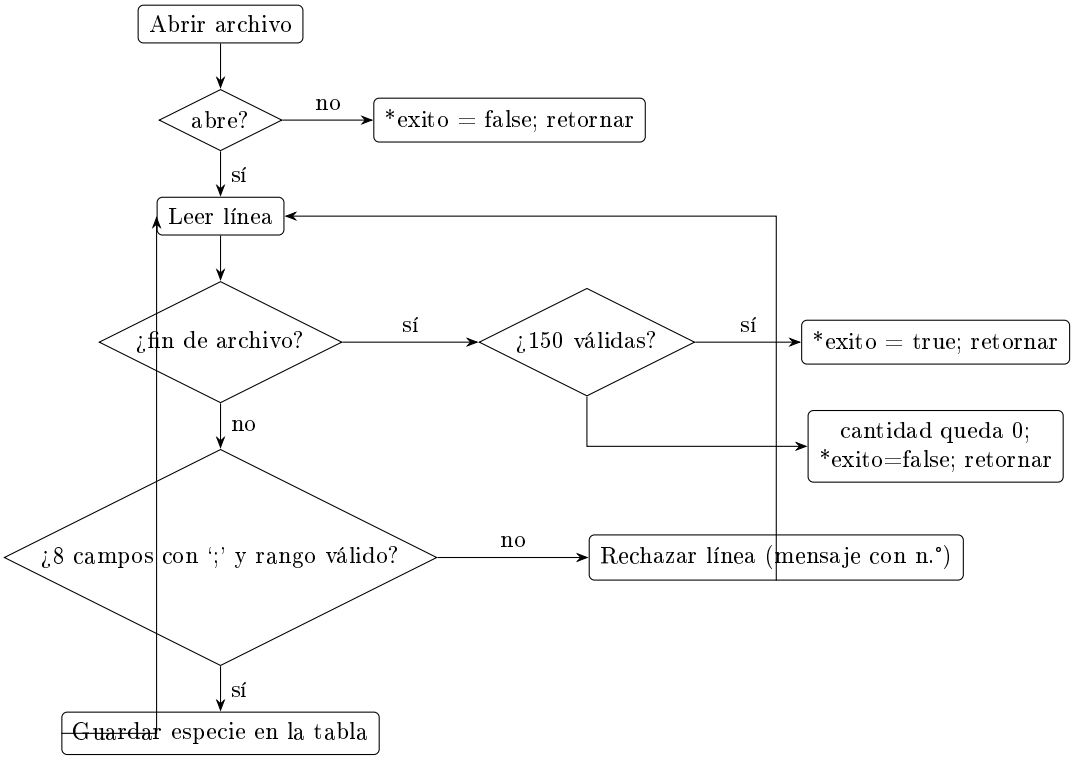
3.2. Inventario y justificación

Estructura	Dónde	Por qué
Arreglo estático `Pokedex[150]`	pokedex	Tamaño fijo conocido (150 especies); acceso directo por número.
Matriz `float[18][18]` de efectividad	tipos (`static` en el `.c`)	Acceso $O(1)$ a cada par de tipos; encapsulada tras `tipos_multiplicador`.
Arreglo `RegistroEntrenadores[32]`	entrenador	Máximo del torneo fijo (D10: exactamente 32).
Lista enlazada de `Ejemplar`	equipo	Tamaño variable (1..6), se crean/destruyen ejemplares dinámicamente; demuestra el TDA pedido.
`Combate[64]`	torneo	El torneo tiene exactamente 64 combates numerados (48 grupos + 16 eliminatoria).
`id_clasificados[16]`	torneo	Los 2 clasificados de cada uno de los 8 grupos (1A, 2A, ..., 1H, 2H).
`filas[4][7]` (salida)	torneo	Tabla de clasificación de un grupo para pantalla/archivo.
`especies_permitidas[150]`	equipo	Restricción de integrantes del backtracking.
Pila implícita (recursión)	equipo (`bt_rec`)	El backtracking es naturalmente recursivo: el estado parcial vive en el stack de llamadas.

4. Diagramas de flujo

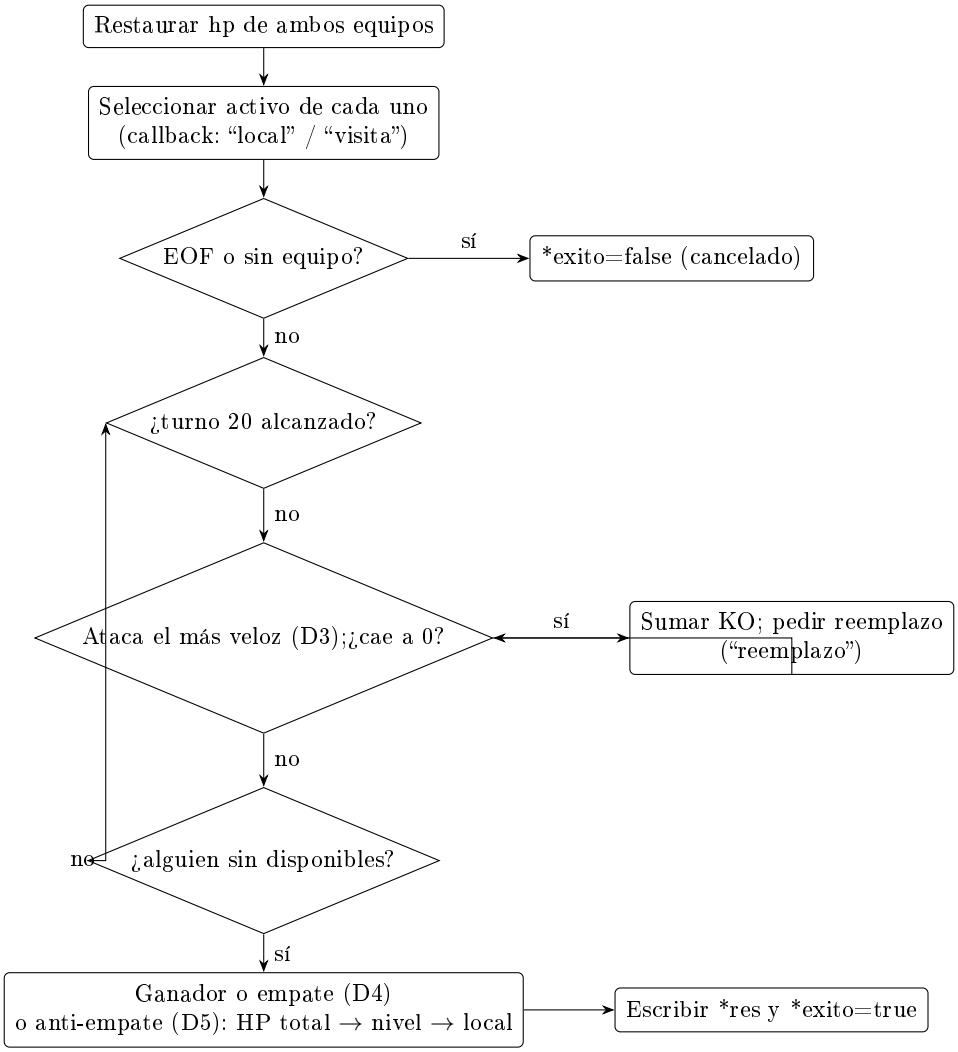
Se presentan los cuatro flujos centrales como diagramas y, además, el flujo resumido de cada módulo.

4.1. Carga de la Pokédex (`pokedex_cargar`)



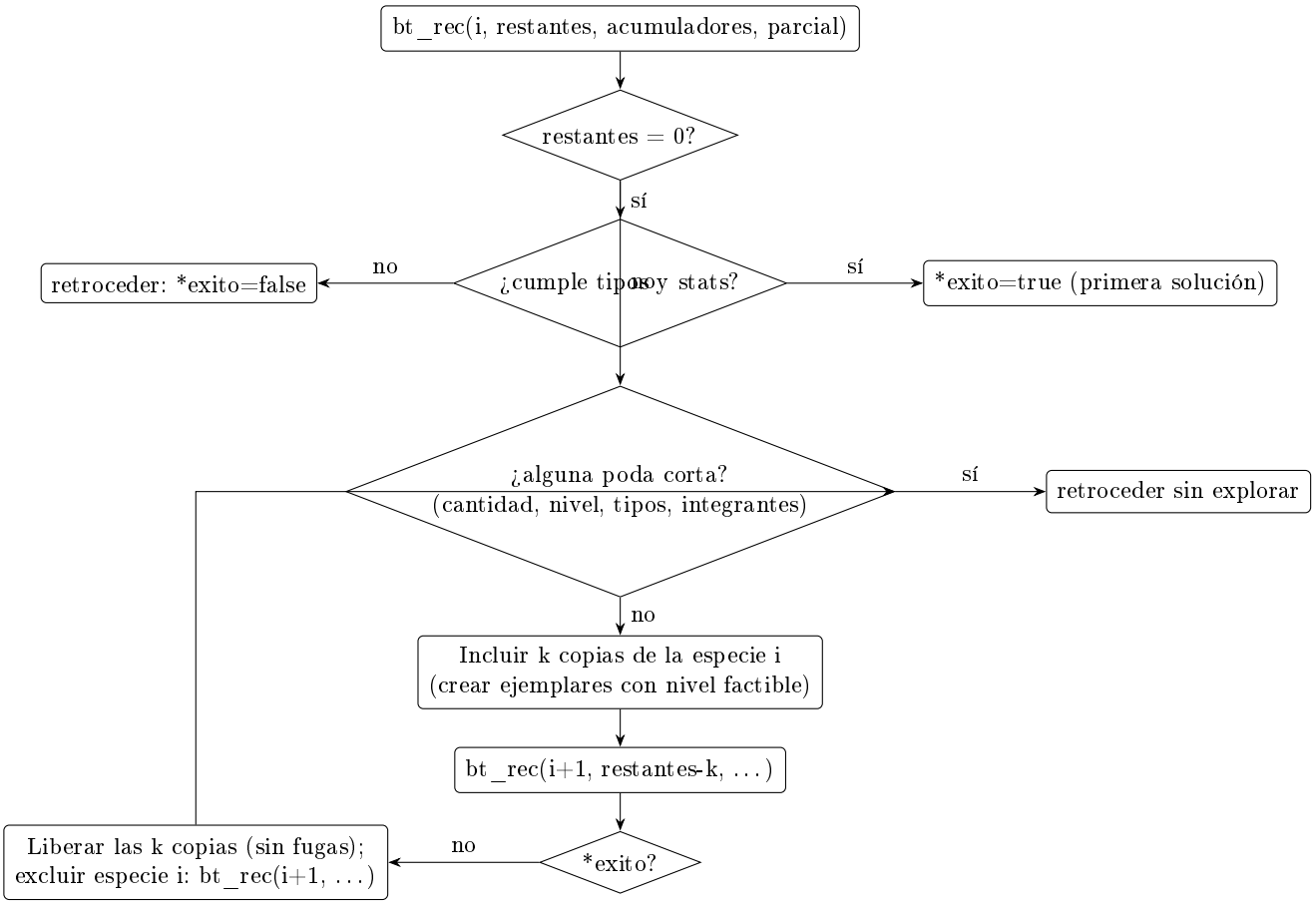
El archivo mal formado deja la Pokédex vacía y el programa continúa (RF-PDX-06); un combate jamás modifica estos datos (RF-PDX-04).

4.2. Combate (`combate_ejecutar`)



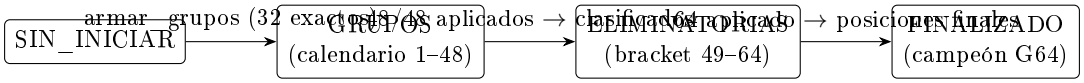
El daño usa la fórmula D1 y el multiplicador de tipos ($\times 2 / \times 1 / \times 0{,}5 / \times 0$, producto si el defensor tiene dos tipos).

4.3. Backtracking de formación de equipos (`bt_rec`)



Las cinco podas (cantidad, cota inferior de nivel, cota de tipos con sufijo, tipos por ejemplar restante y restricción de integrantes) hacen que el caso imposible (6 ejemplares, nivel total máximo 5) se descarte en microsegundos. Sin solución, se informa y *no* se devuelve equipo inválido (RF-EQP-04).

4.4. Máquina de estados del torneo



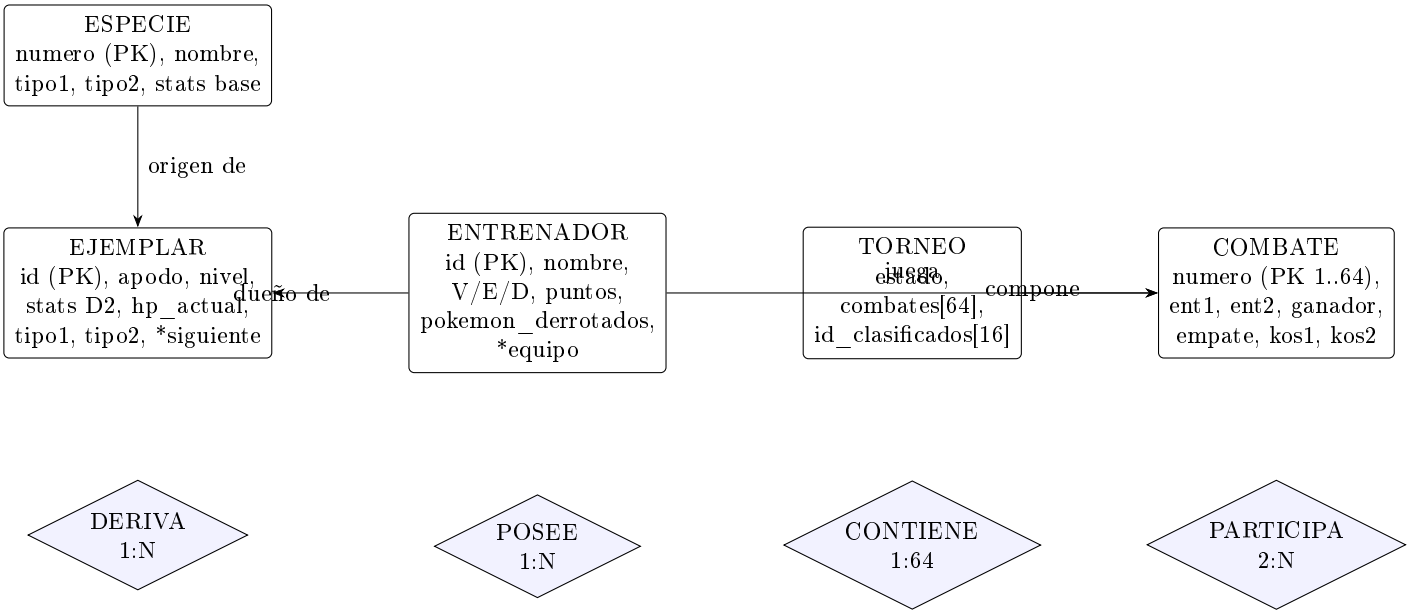
Los participantes de cada combate los resuelve el sistema (RF-RES-03); el usuario nunca los inventa: 49 = 1A-2B, ..., 57 = G49-G50, ..., 63 = P61-P62, 64 = G61-G62.

4.5. Flujo resumido de cada módulo

Módulo	Flujo principal
`pokedex`	cargar (archivo → parseo estricto → tabla) → consultas de solo lectura (mostrar, buscar por número/nombre, ficha).
`tipos`	inicializar (cargar matriz o respaldo embebido) → validar nombres → consultar multiplicador.
`entrenador`	registrar (validar id → rechazar duplicado → insertar) / buscar / listar.
`equipo`	crear ejemplar (especie → fórmula D2) → agregar (lista) → validar; o formar automáticamente (restricciones → backtracking → asignar).
`combate`	restaurar → seleccionar → turnos (daño D1, KO, reemplazo) → fin (empate/anti-empate) → escribir resultado.

`torneo`	armar grupos → calendario → aplicar resultados → ordenar/desempates → clasificados → bracket → campeón.
`resultados`	leer (teclado/archivo) → validar contra el estado real → aplicar o rechazar con motivo.
`archivos`	guardar/cargar entrenadores, resultados y clasificación con los formatos D8 (separador ';').
`validacion`	leer → reintentar ante entrada inválida → devolver valor o EOF (cierre ordenado).
`main`	cargar datos → menú → despachar a módulos → guardar si hubo cambios → liberar memoria → salir.

5. Modelo entidad-relación



Relación	**Cardinalidad**	**Implementación en C**
ESPECIE–EJEMPLAR	1:N (derivación)	El ejemplar referencia la especie por `numero_especie` y *copia* nombre, tipos y stats (fórmula D2) — la especie jamás se modifica.
ENTRENADOR–EJEMPLAR	1:N (composición)	Lista enlazada: `Entrenador.equipo` apunta a la cabeza; cada `Ejemplar.siguiente` al próximo.
TORNEO–COMBATE	1:64 (composición)	Arreglo `Combate[64]` dentro de `Torneo`; numeración fija 1–48 grupos, 49–64 eliminatoria.
COMBATE–ENTRENADOR	2:N (participación)	Cada combate guarda `id_entrenador1/2` resueltos por el sistema, ganador y KOs.
RESULTADO–COMBATE	1:1 (aplicación)	`ResultadoCargado` valida y aplica sobre el combate correspondiente; parciales permitidos.

ARCHIVOS	persistencia externa	`data/*.txt`: pokédex (solo lectura), entrenadores, resultados y clasificación (salida).

6. Cumplimiento del documento de requerimientos

Verificación contra *Proyecto- 2026.pdf*, sección por sección. Evidencia ejecutable: `bash tests/run_tests.sh` (23 PASS / 0 FALLA), `bash scripts/verificar_entrega.sh` (7 PASS) y build con `gcc -std=c99 -Wall -Wextra` con cero warnings.

Requisito (sección)	**Estado**	**Evidencia**
1.1 Pokédex 150 especies, 18 tipos, inmutable	✓	`data/pokedex.txt` (150 líneas, 0 duplicados); API `const`; sha256 intacto tras consultas y combates.
1.2 Consulta de la Pokédex	✓	Opción 1: mostrar todas, buscar por número y por nombre (insensible a mayúsculas/acentos), ficha completa.
1.3 Entrenadores (id único, V/E/D, puntos)	✓	Opción 2 con rechazo de duplicados; struct completo.
1.4 Equipos y ejemplares (separación especie/ejemplar)	✓	`equipo_crear_ejemplar` (fórmula D2 con variación por id); la especie no se muta.
1.5 Backtracking	✓	Opción 3 automática; 5 podas; caso sin solución informa sin equipo inválido.
1.6 Combates	✓	Motor completo: velocidad, daño, KO, reemplazo, agotamiento.
1.7 Efectividad de tipos	✓	Matriz 18×18; valores $\times 2/\times 1/\times 0{,}5/\times 0$ y producto para dos tipos.
1.8 Fase de grupos	✓	8 grupos de 4 (32 exactos), round-robin 1–48 sin repetidos.
1.9 Puntuación y desempates	✓	3/1/0; cadena puntos → victorias → KOs → directo (D9) → id.
1.10 Clasificación automática	✓	1A/2A… 1H/2H resueltos por el sistema al completar 48/48.
1.11 Eliminatorias 49–64	✓	Bracket fijo; G#/P# encadenados; sin empates (anti-empate D5); campeón G64; top 4.
1.12/1.16 Resultados (teclado/archivo, parciales)	✓	Opción 6; `data/resultados.txt` parcial recargable.
1.13 Validación de resultados	✓	Catálogo completo: existan, pertenezcan, ronda correcta, clasificados, eliminados no reaparecen, ganador válido, sin empates, coherencia entre rondas.
1.14 Archivo de la Pokédex	✓	150 especies leídas del archivo; cero datos de especies en el código.
1.15/1.17 Archivos de entrenadores/clasificación	✓	Guardar/cargar con equipos reconstruibles; `clasificacion.txt` con formato exacto.
2. Menú principal	✓*	Las 12 opciones visibles; *ver observación.*
3. Implementación (funciones, structs, arreglos/dinámicas, archivos, cadenas, TDA, recursividad, backtracking, módulos, main flujo)	✓	10 módulos; lista enlazada; `bt_rec` recursivo; main sin lógica de negocio.

4. Validación de entradas	✓	`validacion.c`; ninguna entrada termina el programa inesperadamente (batería E2E).
6. Pruebas del sistema (16 puntos)	✓	Cobertura 16/16 en `tests/` (ver trazabilidad).
7. Entrega (un comprimido, compila/ejecuta sin externos)	✓	`make entrega` → zip de 93 archivos; `verificar_entrega.sh`: 7 PASS en directorio limpio.
8. Evaluación (13 criterios + defensa)	✓	Cada criterio mapeado en `docs/trazabilidad-requisitos.md`; este documento prepara la defensa.

Observación honesta (la única): la opción 9 del menú ("Consultar historial de combates") se muestra pero responde "en construcción". El enunciado (sección 2) permite modificar el menú; se recomienda, antes de la entrega, implementarla (registro de combates de la sesión) o retirarla del menú para no dejar funciones sin terminar.

7. Guion de defensa: preguntas probables

1. **¿Por qué tantas funciones `static` en `main.c`?** Enlace interno: privadas del archivo, evitan colisiones de nombres y encapsulan (es el `private` de C). Se demuestra con `nm`: los `static` son símbolos locales `t` y los públicos `T`.

2. **¿Por qué la Pokédex nunca se modifica?** Separación especie/ejemplar (criterio central): el ejemplar copia lo que necesita y solo muta `hp_actual`; las funciones de consulta son `const`-correctas y el sha256 del archivo queda intacto.

3. **¿De dónde salen las estadísticas de un ejemplar?** Fórmula propia documentada (D2): `(base*nivel/50)+nivel+5/10` más variación determinista `(id*7)%16` en [0,15]: variabilidad entre ejemplares y pruebas 100 % reproducibles (libertad otorgada por el docente).

4. **¿Por qué no usaste la fórmula oficial de daño?** La oficial incluye IVs/EVs y el término del poder del movimiento (fuera de alcance); sin él, los combates serían interminables. La fórmula D1 usa el mínimo exigido (nivel, ataque, defensa, tipos) y está justificada en el informe.

5. **¿Cómo evita el backtracking explotar con 150 especies?** Cinco podas (cantidad, cota inferior de nivel, cota de tipos, tipos por ejemplar restante e integrantes permitidos); el caso imposible se descarta en microsegundos y no devuelve equipo inválido.

6. **¿Qué estructuras de datos usaste y por qué?** Arreglos estáticos donde el tamaño es fijo (Pokédex 150, 32 entrenadores, 64 combates), matriz 18×18 para efectividad y lista enlazada para los equipos (tamaño variable). No hay pila ni cola explícitas: la recursión del backtracking usa la pila del sistema.

7. **¿Cómo validas que un resultado del torneo es correcto?** Ocho chequeos (existen, pertenecen al combate, ronda correcta, clasificados, eliminados, ganador válido, sin empates en eliminatoria, coherencia entre rondas) y los participantes siempre los resuelve el sistema.

8. **¿Por qué `main` no tiene lógica de negocio?** Cada módulo es un TDA; `main` solo despacha. El combate ni siquiera lee la consola: recibe un callback (`CombateSeleccionar`) que `main` provee — por eso los probes pueden probar combates sin teclado.

9. **¿Qué convención siguen tus funciones?** RF-CNV-01..03: cada función devuelve un solo resultado; las operaciones con varias salidas son procedimientos `void` con parámetros por referencia (p. ej. `void pokedex_cargar(..., bool *exito)`).

8. Comandos útiles durante la defensa

- `make` / `make run` / `make clean` — compilar, ejecutar, limpiar.
- `bash tests/run_tests.sh` — batería completa (23 PASS / 0 FALLA).
- `bash scripts/verificar_entrega.sh` — compilar y ejecutar el zip en un directorio limpio.
- `make entrega` — regenerar el paquete `dist/gran-torneo-pokemon-entrega.zip`.
- `nm build/torneo | grep -E "[tT] "` — símbolos locales (`t`, static) vs globales (`T`).
- `doxygen Doxyfile` — documentación HTML del código (binario local en `tools/` o `sudo apt install doxygen`).
- `grep -h "### Requirement.°penspec/specs/*/spec.md` — los 45 requisitos formales con su código.

Documento generado como material de estudio para la defensa oral del proyecto **El Gran Torneo Pokémon** (Programación I, Universidad de Carabobo).